04 — Integração TDD + BDD + DDD

Requisito escolhido: **12 — Transferir valores entre contas**.

Com ele dá para mostrar as três abordagens sobre a mesma funcionalidade, cada uma em um nível diferente.

```
BDD  transferir_valores.feature        "o que o sistema deve fazer"   (visão do usuário)
 │     └─ ContaSteps → SistemaBancario.transferir(idOrigem, idDestino, valor)
 │                        └─ GerenciadoraContas.transfereValor  (localiza as contas)
DDD                           └─ ContaCorrente.transferir(destino, valor)  (regra do
domínio)
TDD  TransferenciaTest                  "como implementar e validar"    (visão do código)
```

TDD

Os testes de `test/java/tdd/TransferenciaTest.java` conduziram a implementação de `ContaCorrente.transferir`, **uma regra por vez** (detalhes e saídas reais em `01-tdd.pdf`):

Ordem	Teste	O que obrigou a implementar
1	`deveTransferirQuandoExisteSaldoSuficiente`	debitar a origem e creditar o destino
2	`naoDeveTransferirQuandoSaldoForInsuficiente`	verificar `saldo >= valor`
3	`naoDeveTransferirValorNegativo`	rejeitar `valor < 0` (defeito real do código original)
4	`naoDeveTransferirValorZero`	rejeitar `valor == 0` (`<` virou `<=`)
5 (auditoria)	`naoDeveTransferirParaDestinoNulo`	verificar o destino **antes** do débito
6 (auditoria)	`naoDeveTransferirQuandoSaldoDoDestinoDeixariaDeSerFinito`	recusar crédito que gere `Infinity`
7 (2ª auditoria)	`SaldoFinitoTest` (3 testes)	invariante da conta: saldo sempre finito (construtor e `setSaldo`)

Cada teste falhou antes (RED), passou com a mudança mínima (GREEN) e, nos refactors, o código ganhou nomes do domínio (`possuiSaldoSuficiente`, `debitar`, `creditar`, `valorPositivo`). No fim, o repositório passou a delegar para a entidade.

BDD

Arquivo: `test/resources/features/transferir_valores.feature` — steps em `test/java/bdd/ContaSteps.java`.

Como cliente do banco **Quero** transferir valores da minha conta corrente para outra conta **Para** movimentar meu dinheiro sem precisar sacar e depositar

```
Contexto:
  Dado que existe uma conta corrente cadastrada com id 1 e saldo 100,00
  E que existe uma conta corrente cadastrada com id 2 e saldo 0,00
```

Transferência realizada com sucesso

```
Quando for solicitada a transferência de 40,00 da conta 1 para a conta 2
Então a transferência deverá ser realizada
E o saldo da conta 1 deverá ser 60,00
E o saldo da conta 2 deverá ser 40,00
```

Ou seja: dado que uma conta possui saldo suficiente e existe uma conta de destino, quando uma transferência válida é realizada, o valor é debitado da origem e creditado no destino.

Transferência sem saldo suficiente

```
Quando for solicitada a transferência de 150,00 da conta 1 para a conta 2
Então a transferência deverá ser rejeitada
E o saldo da conta 1 deverá ser 100,00
E o saldo da conta 2 deverá ser 0,00
```

Ou seja: a operação é rejeitada e os saldos permanecem inalterados.

Transferência com valor inválido (Esquema do Cenário)

```
Quando for solicitada a transferência de <valor> da conta 1 para a conta 2
Então a transferência deverá ser rejeitada
E o saldo da conta 1 deverá ser 100,00
E o saldo da conta 2 deverá ser 0,00

Exemplos:
  | valor  |
  | 0,00   |
  | -50,00 |
```

Como o BDD dirigiu o código (de fora para dentro): o cenário foi escrito usando uma operação que ainda não existia na fachada, `SistemaBancario.transferir(idOrigem, idDestino, valor)`. A primeira execução falhou (`evidencias/30-integracao-bdd-transferencia-red.txt`):

```
[ERROR] .../bdd/ContaSteps.java:[57,49] cannot find symbol
[ERROR]   symbol:   method transferir(int,int,double)
[INFO] BUILD FAILURE
```

Foi criada a operação (uma linha delegando ao repositório) e os cenários passaram (`evidencias/31-integracao-bdd-transferencia-green.txt`):

```
14 Scenarios (14 passed)
63 Steps (63 passed)
```

Os 14 cenários incluem os 10 do doc 02 e os 4 de transferência (o Esquema gera 2 cenários).

DDD

Conceito	Onde está representado	Responsabilidade
ContaCorrente (entidade)	`negocio.ContaCorrente`	Tem identidade (`id`) e guarda o próprio saldo
Saldo	campo privado `saldo` de `ContaCorrente`	Só é alterado por `debitar`/`creditar` (privados) durante a transferência. O `setSaldo` original foi mantido por compatibilidade.
Transferência	`ContaCorrente.transferir(destino, valor)`	A conta de origem executa a operação sobre a conta de destino
Regra de saldo suficiente	`ContaCorrente.possuiSaldoSuficiente(valor)`	Pergunta feita à própria conta
Valor válido	`ContaCorrente.valorPositivo(valor)`	Transferência só com valor > 0 (rejeita também `NaN`)
Destino válido / crédito possível	`transferenciaPermitida` / `podeReceberCredito`	Rejeita destino nulo e crédito que tornaria o saldo infinito
Débito e crédito	`ContaCorrente.debitar` / `creditar` (privados)	Ninguém de fora altera o saldo "no meio" de uma transferência
Localizar as contas	`GerenciadoraContas.transfereValor(idOrigem, valor, idDestino)`	Repositório: encontra as duas entidades pelo id e devolve `false` se alguma não existe
Caso de uso	`SistemaBancario.transferir(idOrigem, idDestino, valor)`	Fachada usada pelo cenário BDD

Antes, o repositório fazia `contaOrigem.setSaldo(contaOrigem.getSaldo() - valor)`: a regra ficava **fora** da conta, e foi por isso que o defeito do valor negativo passou despercebido. Agora:

```
public boolean transferir(ContaCorrente destino, double valor) {
    if (!transferenciaPermitida(destino, valor))
        return false;

    this.debitar(valor);
    destino.creditar(valor);
    return true;
}

private boolean transferenciaPermitida(ContaCorrente destino, double valor) {
    return destino != null
            && valorPositivo(valor)
            && possuiSaldoSuficiente(valor)
            && destino.podeReceberCredito(valor);
}
```

O código pode ser lido em voz alta como a regra de negócio: "*se a transferência é permitida (há destino, o valor é positivo, há saldo suficiente e o destino pode receber o crédito), debita a origem e credita o destino*". Todas as condições são verificadas **antes** do débito; as duas últimas (destino nulo e saldo não finito) foram acrescentadas após a auditoria (`06-correcoes-pos-auditoria.pdf`).

Síntese

Abordagem	Pergunta que responde	Artefato no requisito 12
BDD	**O que** o sistema deve fazer?	`transferir_valores.feature` (legível pelo cliente/professor)
DDD	**Como representar** corretamente o domínio?	`ContaCorrente.transferir`, `possuiSaldoSuficiente`, `debitar`, `creditar`
TDD	**Como implementar e validar** incrementalmente?	`TransferenciaTest`: 4 ciclos RED → GREEN → REFACTOR + 2 após a auditoria

> **BDD** = o que o sistema deve fazer **DDD** = como representar corretamente o domínio **TDD** = como implementar e validar incrementalmente